

Aulas Dia 01 - VitrineCol

Compilação Técnica Editorial

Sumário

Capítulo 01 - Aula 1 – Backlog do MVP a partir das personas e formação das equipes

Capítulo 02 - Aula 2 – Git e GitHub: repositório, commits, README e quadro Kanban

Capítulo 03 - Aula 3 – HTML: estrutura da página, textos, links e imagens, com o Live Server

Capítulo 01: Aula 1 – Backlog do MVP a partir das personas e formação das equipes

Origem: dia01-aula01-backlog-do-mvp-e-formacao-das-equipes.md

Aula 1 – Backlog do MVP a partir das personas e formação das equipes

Dia 1 · Ter 06/10/2026 · Aula 1 · UC3

- **Requisitos cobertos:** visão geral dos 25 requisitos funcionais (RF-01 a RF-25), só leitura: nesta aula você apenas lê e prioriza a lista, sem programar nada
- **Tempo:** 10 minutos de abertura, 40 de prática e 10 de fechamento (60 minutos)
- **Ponto de partida:** nenhum arquivo ainda. Você só precisa de um editor de texto ou de uma planilha para anotar (Google Docs, Google Planilhas, Word, Excel ou o Bloco de Notas)

Esta é a primeira aula do curso, então não existe aula anterior para refazer. Se algum passo ficar incompleto, termine-o em casa antes da Aula 2.

Objetivo da aula

Em 60 minutos você vai transformar um problema e duas personas em uma **lista priorizada de funcionalidades** (o backlog do MVP) e combinar com a sua equipe quem faz o quê.

Abertura (10 minutos). Como esta é a primeira aula de programação do projeto, a "retomada" é sobre o que você já conhece: nas aulas anteriores do curso a turma definiu o problema (as lojas de roupa do bairro vendem pouco pela internet e as clientes não conseguem comparar peças de lojas diferentes) e as personas. Leia o resumo abaixo em voz alta com a sua equipe e confirme que todas entendem o que será construído.

O produto, em uma frase: uma plataforma web de moda local em que a cliente filtra roupas por tipo, tamanho e loja, junta peças de lojas diferentes na mesma sacola e faz

um pedido por loja, combinado pelo WhatsApp; a lojista cadastra a loja e os produtos (até 5 fotos cada) e responde os pedidos.

Ao final você terá: (1) o backlog dos 25 requisitos com prioridade, (2) a equipe formada com papéis combinados e (3) um combinado de como a equipe vai trabalhar.

O Conceito

Termos desta aula

- **Persona:** uma pessoa inventada, mas parecida com quem vai usar o sistema, com nome, rotina e necessidades. Ajuda a pensar "o que a Marina precisa?" em vez de "o que eu acho legal?".
- **Backlog:** a lista de tudo o que o sistema precisa fazer, em ordem de importância. É a "lista de compras" do projeto.
- **Requisito funcional (RF):** uma frase que diz o que o sistema deve fazer, por exemplo "listar os produtos em cards". Neste curso cada um tem um código: RF-01, RF-02 e assim por diante.
- **MVP:** a menor versão do sistema que já entrega o fluxo completo de ponta a ponta (aqui: escolher peças, montar a sacola e fazer o pedido). Funcionalidades que não são indispensáveis ficam para depois.

Por que começar por uma lista? Pense em fazer uma viagem: antes de arrumar a mala você decide o destino e anota o que não pode faltar. Um projeto de software é igual: sem lista, a equipe gasta o tempo em detalhes bonitos e esquece o essencial. Com a lista priorizada, quando o tempo apertar você já sabe o que pode ficar de fora.

Três níveis de prioridade usados neste curso:

Prioridade	Significa	Se faltar tempo
Essencial	sem isso o sistema não funciona (não existe MVP)	nunca sai
Importante	melhora muito a experiência	fica pronto ou é registrado como "fora do escopo"
Desejável	é bom ter, mas o sistema funciona sem	é o primeiro a sair

Mão na Massa

Passo 1: leia o problema e as duas personas (10 minutos)

Se a sua equipe já tem personas feitas nas aulas anteriores, use as suas. Se não tiver, use estas duas:

- **Marina, 22 anos, estudante.** Gosta de moda, mas não tem tempo de andar por várias lojas. Quer ver o que cada loja do bairro tem, comparar preços, separar as peças e combinar a retirada pelo WhatsApp, que ela já usa todo dia. Mexe pouco no computador e usa muito o celular.
- **Dona Célia, 48 anos, dona de uma loja de roupas.** Vende só no balcão e no Instagram. Quer mostrar os produtos com várias fotos, controlar o estoque por tamanho e receber os pedidos de forma organizada, sem perder mensagens. Precisa de telas simples, com poucos botões.

Passo 2: transforme necessidades em funcionalidades (15 minutos)

Em equipe, complete cada frase abaixo no formato "**Como [persona], quero [o quê], para [por quê]**". Escreva pelo menos 6 frases. Exemplos para você começar:

1. Como **Marina**, quero filtrar as roupas por tipo, tamanho e loja, para achar rápido o que combina comigo.
2. Como **Marina**, quero juntar peças de lojas diferentes na mesma sacola, para fazer a compra de uma vez.
3. Como **Dona Célia**, quero cadastrar a minha loja e os meus produtos com fotos, para as clientes verem o que eu vendo.
4. Como **Dona Célia**, quero ver os pedidos que chegaram e avisar a cliente quando eu confirmar, para ela saber o que está acontecendo.

Acrescente as suas próprias frases (por exemplo: recuperar a senha esquecida, ver a loja no mapa, apagar a própria conta).

Passo 3: compare com a lista oficial dos 25 requisitos (15 minutos)

Abaixo está a lista que vamos construir durante o curso. Copie a tabela para o seu documento e **marque ao lado de cada linha qual frase do Passo 2 ela atende**. Se alguma frase sua não aparecer na lista, anote como "ideia para depois": ela não entra no MVP.

Código	O sistema deve	Prioridade
RF-01	Listar os produtos ativos em cards (foto de capa, nome, preço e loja), 12 por página, com o botão "Carregar mais produtos"	Essencial
RF-02	Filtrar o catálogo por tipo de roupa	Essencial
RF-03	Filtrar o catálogo por loja	Essencial
RF-04	Filtrar o catálogo por tamanho, mostrando só produtos com estoque nesse tamanho	Importante
RF-05	Buscar produtos pelo nome, sem diferenciar maiúsculas de minúsculas	Desejável
RF-06	Mostrar a página da loja com nome, descrição, endereço, WhatsApp e produtos	Essencial
RF-07	Oferecer um botão com o link do mapa da loja	Desejável
RF-08	Mostrar a página do produto com galeria de fotos, descrição, preço, loja e tamanhos, com escolha de tamanho e quantidade	Essencial
RF-09	Manter uma sacola com peças de várias lojas: adicionar, mudar quantidade, remover, subtotal por loja e total	Essencial
RF-10	Finalizar a sacola de uma cliente logada, gravando um pedido por loja e mostrando um botão de WhatsApp por loja	Essencial
RF-11	Mostrar à cliente os pedidos dela, agrupados por compra, com o status e o recado da loja	Importante
RF-12	Cadastrar usuária com nome, e-mail, senha, perfil e telefone opcional	Essencial
RF-13	Permitir entrar e sair da conta, mantendo a sessão	Essencial
RF-14	Controlar o acesso às telas conforme o perfil (cliente, lojista ou visitante)	Essencial
RF-15	Permitir à lojista criar e editar a própria loja	Essencial
RF-16	Permitir à lojista cadastrar produto com nome, descrição, categoria, preço e tamanhos com estoque	Essencial

Código	O sistema deve	Prioridade
RF-17	Permitir à lojista editar produto	Essencial
RF-18	Permitir à lojista desativar ou excluir produto	Essencial
RF-19	Enviar até 5 fotos por produto (JPG, PNG ou WebP, até 2 MB), escolher a capa e reordenar	Importante
RF-20	Mostrar à lojista os pedidos da loja e permitir mudar o status e deixar um recado	Importante
RF-21	Avisar carregamento, erro e sucesso em toda chamada ao banco, sem deixar tela em branco	Essencial
RF-22	Avisar a cliente, dentro do sistema, quando a lojista mudar o status ou deixar um recado	Importante
RF-23	Mostrar uma página inicial de vitrine (hero, carrossel, tipos de roupa, destaques e lojas)	Importante
RF-24	Permitir recuperar a senha por e-mail	Importante
RF-25	Permitir à usuária excluir a própria conta	Importante

Quando todas as linhas estiverem conferidas, a equipe responde em voz alta: **"Se faltar tempo, o que sai primeiro?"** A resposta combinada para o curso é, nesta ordem: (1) o link do mapa da loja, (2) o carrossel da página inicial e a reordenação das fotos, (3) o filtro por tamanho, (4) o envio de fotos pelo sistema.

Passo 4: forme a equipe e combine os papéis (10 minutos)

Cada equipe tem de 4 a 5 pessoas. Existem cinco papéis, que **rodam** ao longo do curso para todas aprenderem um pouco de cada área:

Papel	O que faz
Produto/negócio	Cuida do backlog e das prioridades; decide o que entra e o que sai
UX/UI	Cuida das telas, das cores e da facilidade de uso
Front-end	Escreve o HTML, o CSS e o JavaScript das telas
Banco/back-end	Cuida das tabelas, das regras de acesso e da ligação com o Supabase
Conteúdo/QA	Cuida dos textos, das fotos e dos testes

Anote em uma tabela quem assume cada papel **nesta semana** e quando o papel muda (sugestão: troca a cada dia de aula). Em programação, **todas escrevem código**: o papel só diz quem "puxa" a conversa daquela área. Combine também que as duplas de trabalho se alternam (uma digita, outra confere).

Passo 5: combinados da equipe (5 minutos)

Escreva no documento, em poucas linhas: como a equipe se comunica (grupo de mensagens), em que horário se encontra fora da aula, e que **ninguém fica com dúvida sozinha por mais de 15 minutos**: pede ajuda à equipe.

Explicação do Código

Esta aula não tem código, então aqui explicamos cada decisão:

- **Por que usar personas?** Elas obrigam a equipe a pensar em pessoas reais. A Marina usa o celular; por isso, no Dia 5, as telas serão feitas primeiro para o celular. A Dona Célia quer telas simples; por isso o painel da lojista terá poucas telas.
- **Por que o formato "Como..., quero..., para..."?** Ele força a equipe a escrever o "por quê". Se você não consegue explicar o motivo, talvez a funcionalidade não seja necessária.
- **Por que 25 requisitos e não mais?** O curso tem poucas semanas. A lista foi cortada para caber: pagamento online, entrega, avaliações, mapa embutido e aplicativo de celular ficam de fora de propósito.
- **Por que marcar "Essencial", "Importante" e "Desejável"?** Para decidir sem brigar quando o tempo acabar. Quem escolheu o corte foi a lista, e não a pessoa.

- **Por que os papéis rodam?** Quem só faz o banco nunca aprende a tela. No fim do curso todas devem entender o sistema inteiro.

Validação

Confira com a sua equipe, marcando cada item:

- [] Existe um documento com pelo menos 6 frases "Como..., quero..., para..." e a tabela dos 25 requisitos com a coluna "frase que atende".
- [] Todos os requisitos **Essenciais** estão marcados e a equipe sabe nomeá-los em voz alta (RF-01, 02, 03, 06, 08, 09, 10, 12, 13, 14, 15, 16, 17, 18 e 21).
- [] Os cinco papéis têm responsável nesta semana.
- [] A equipe sabe responder: "o que sai primeiro se faltar tempo?".

Erros comuns

1. *Sintoma:* a lista ficou com 60 itens e ninguém sabe o que fazer primeiro. *Causa:* cada ideia virou um requisito. *Correção:* junte os itens parecidos e mova o que não está na lista dos 25 para a coluna "ideias para depois".
2. *Sintoma:* todos os requisitos ficaram "Essenciais". *Causa:* medo de deixar algo de fora. *Correção:* pergunte "o sistema funciona sem isso?". Se a resposta for sim, não é essencial.
3. *Sintoma:* só uma pessoa fez a lista inteira. *Causa:* o grupo esperou uma pessoa começar. *Correção:* divida os 25 requisitos entre as integrantes (5 para cada uma), cada uma marca a sua parte e depois todas revisam juntas.

Se travar

1. Releia o Passo em que parou e confira se o exemplo dado ainda não resolve a sua dúvida.
2. Compare o seu documento com a tabela da aula: todos os códigos de RF-01 a RF-25 estão lá?
3. Se ainda assim travar, volte ao último passo que funcionou e refaça-o com a equipe. Só depois peça ajuda à sua equipe, dizendo o passo e o que você já tentou.

Seu projeto agora tem

- Um documento de backlog com os 25 requisitos priorizados (ainda sem repositório e sem código).

- A equipe formada, com papéis combinados e a regra "se faltar tempo, o que sai primeiro".

Como saber que deu certo: a sua equipe consegue explicar, sem olhar a tabela, o que o sistema faz e quais funcionalidades nunca podem sair do projeto.

Capítulo 02: Aula 2 – Git e GitHub: repositório, commits, README e quadro Kanban

Origem: dia01-aula02-git-e-github-repositorio-commits-e-quadro-kanban.md

Aula 2 – Git e GitHub: repositório, commits, README e quadro Kanban

Dia 1 · Ter 06/10/2026 · Aula 2 · UC3

- **Requisitos cobertos:** não se aplica (preparação do ambiente de desenvolvimento)
- **Tempo:** 10 minutos de abertura, 40 de prática e 10 de fechamento (60 minutos)
- **Ponto de partida:** o backlog e a equipe da Aula 1. Você precisa do VS Code instalado e de uma conta gratuita no GitHub (ou criá-la agora)

Se você não terminou a aula anterior, refaça os passos dela (estão na seção Mão na Massa) antes de começar esta.

Objetivo da aula

Em 60 minutos você vai instalar o Git e a extensão Live Server, criar o **repositório** da equipe no GitHub, fazer o seu primeiro **commit** e montar o **quadro Kanban** com o backlog da Aula 1.

Abertura (10 minutos). Retomada: na Aula 1 vocês montaram o backlog (a lista dos 25 requisitos). Hoje essa lista vai para um quadro visual, e o código do projeto passa a ter um "histórico de versões" compartilhado. Confira antes de começar: o backlog está em um documento e a equipe sabe quem assume cada papel.

O Conceito

Termos desta aula

- **Git:** programa que guarda o histórico de todas as versões dos seus arquivos. Funciona como o "desfazer" mais poderoso que existe: você volta a qualquer momento ao que estava salvo antes.
- **Commit:** uma "foto" do projeto em um momento, com uma mensagem que explica o que mudou. Cada commit deve ser pequeno e ter uma mensagem em português, no imperativo: "Adiciona o formulário de login".
- **Repositório:** a pasta do projeto com todo o histórico de commits. Existe uma cópia no seu computador e outra no GitHub.
- **GitHub:** site que guarda o repositório na internet para a equipe toda acessar. **push** envia os seus commits para lá; **pull** traz os commits das colegas.
- **Kanban:** quadro com colunas (a fazer, em andamento, concluído) onde cada tarefa é um cartão que anda da esquerda para a direita.

Analogia: o Git é um caderno em que cada commit é uma página datada; o GitHub é o armário da turma onde cada uma guarda uma cópia do caderno. O Kanban é o mural na parede com os posts das tarefas.

Sobre o idioma das ferramentas: o VS Code pode estar em português ou em inglês, dependendo de como foi instalado, e o GitHub continua em inglês (a tela não é traduzida). Por isso, nos passos, o nome do botão ou do menu aparece em inglês, **exatamente como na tela**, e logo depois vem a tradução em português do Brasil para você entender o que está clicando. Se o seu VS Code já está em português, siga a versão em português.

Mão na Massa

Passo 1: confira o Git e instale a extensão Live Server

1. Abra o VS Code. Abra o terminal integrado: no menu **Terminal > New Terminal** (em português: **Terminal > Novo Terminal**). O atalho é Ctrl+` ou Ctrl+J (a tecla do acento grave, no Windows e no Mac).
2. No terminal, digite e aperte Enter:

```
git --version
```

Deve aparecer algo como `git version 2.43.0`. Se aparecer "comando não encontrado": no **Windows**, instale pelo site <https://git-scm.com/download/win> (aceite as opções padrão), feche e abra o VS Code de novo; no **Mac**, o próprio comando oferece instalar as ferramentas de desenvolvimento (clique em **Install**, em português: **Instalar**).

1. Instale o Live Server: clique no ícone de quadradinhos da barra lateral esquerda, **Extensions** (em português: **Extensões**), ou use Ctrl+Shift+X (no Mac, Cmd+Shift+X). Na busca digite `Live Server`, escolha a extensão de **Ritwick Dey** e clique em **Install** (em português: **Instalar**). Quando terminar, aparece o botão **Go Live** no canto inferior direito do VS Code.

Passo 2: diga ao Git quem é você

No terminal (troque pelo seu nome e e-mail; use o mesmo e-mail da conta do GitHub):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

Passo 3: crie o repositório no GitHub (uma integrante por equipe)

Se você estuda sozinha, faça este passo na sua conta e pule o convite do item 4.

1. Entre em <https://github.com> e faça login (se não tiver conta, **Sign up**, em português: **Cadastre-se**).
2. No canto superior direito clique no botão + e depois em **New repository** (em português: **Novo repositório**).
3. Em **Repository name** (em português: **Nome do repositório**) escreva `vitrine-col`. Deixe **Public** (em português: **Público**) marcado. **Não** marque a opção **Add a README file** (em português: **Adicionar um arquivo README**). Clique em **Create repository** (em português: **Criar repositório**).
4. Convide as colegas: **Settings > Collaborators > Add people** (em português: **Configurações > Colaboradores > Adicionar pessoas**) e informe o nome de usuária de cada uma. Elas recebem um e-mail e precisam aceitar o convite.

Passo 4: crie a pasta do projeto e os dois primeiros arquivos (a mesma integrante)

1. No computador, crie uma pasta chamada `vitrine-col` (por exemplo, dentro de Documentos).

2. No VS Code: **File > Open Folder...** (em português: **Arquivo > Abrir Pasta...**) e escolha a pasta `vitrine-col`. Se o VS Code perguntar se você confia nos autores dos arquivos desta pasta, clique em **Yes, I trust the authors** (em português: **Sim, eu confio nos autores**).
3. No painel **Explorer** (em português: **Explorador**), passe o mouse sobre o nome da pasta e clique no ícone **New File** (em português: **Novo Arquivo**). Crie os dois arquivos abaixo, colando o conteúdo e salvando com Ctrl+S (no Mac, Cmd+S).

Arquivo: `README.md` (arquivo novo, inteiro)

```
# Vitrine Col

Plataforma web de moda local feita no curso **Jovem Programadora (Senac)**.

Clientes filtram roupas por tipo, tamanho e loja, juntam peças de lojas diferentes na mesma sacola e fazem um pedido por loja, combinado pelo WhatsApp. Lojistas cadastram a loja e os produtos.

## Integrantes da equipe

- Nome da primeira integrante

## Como rodar

Abra a pasta no VS Code e clique em **Go Live** (extensão Live Server).
```

Arquivo: `.gitignore` (arquivo novo, inteiro)

```
# Arquivos do sistema operacional
.DS_Store
Thumbs.db
desktop.ini

# Arquivos dos editores
.vscode/
.idea/

# Segredos e arquivos locais: nunca versionar
.env
.env.*
*.local

# Registros e temporários
*.log
*.tmp
*.bak
```

Passo 5: faça o primeiro commit e envie para o GitHub

No terminal do VS Code (troque `SEU-USUARIO` pelo nome de usuária do GitHub):

```
git init
git branch -M main
git add .
git commit -m "Cria o README e o .gitignore"
git remote add origin https://github.com/SEU-USUARIO/vitrine-col.git
git push -u origin main
```

Na primeira vez, o Git pode abrir uma janela do navegador pedindo para você entrar no GitHub e autorizar: faça isso e volte ao VS Code. Depois, atualize a página do repositório no GitHub: os dois arquivos devem aparecer.

Passo 6: as outras integrantes clonam e fazem o primeiro commit (uma de cada vez)

1. Cada integrante abre o terminal em uma pasta de trabalho (por exemplo, Documentos) e digita:

```
git clone https://github.com/SEU-USUARIO/vitrine-col.git
```

Depois abre a pasta `vitrine-col` criada: **File > Open Folder...** (em português: **Arquivo > Abrir Pasta...**).

1. Edite o `README.md` e troque a linha `- Nome da primeira integrante` pelo seu nome, acrescentando uma nova linha para cada integrante (uma de cada vez, para não bagunçar).
Salve.
2. Faça o commit e o envio:

```
git add .
git commit -m "Adiciona meu nome ao README"
git pull
git push
```

O `git pull` traz o que as colegas já enviaram antes de você enviar o seu. Se aparecer um conflito (o arquivo mostra trechos entre `<<<<<<`, `=====` e `>>>>>>`), apague essas três marcas e deixe as linhas de todas as pessoas no arquivo; depois faça `git add .`, `git commit -m "Resolve conflito no README"` e `git push`.

Passo 7: monte o quadro Kanban

1. No repositório do GitHub, clique na aba **Projects** (em português: **Projetos**) e depois em **New project** (em português: **Novo projeto**).
2. Escolha o modelo **Board** (em português: **Quadro**), dê o nome `Kanban da equipe` e clique em **Create project** (em português: **Criar projeto**).
3. O quadro vem com as colunas **Todo** (em português: **A fazer**), **In Progress** (em português: **Em andamento**) e **Done** (em português: **Concluído**). Na coluna **Todo**, clique em **Add item** (em português: **Adicionar item**), escreva o título e aperte Enter. Crie **um cartão para cada requisito Essencial** do backlog da Aula 1, por exemplo: `RF-01 Listar os produtos em cards`.
4. Combine na equipe: quem pega um cartão arrasta para **In Progress** (em português: **Em andamento**); quando termina, arrasta para **Done** (em português: **Concluído**).

Explicação do Código

- `git --version`: mostra a versão instalada. Serve só para provar que o Git existe.
- `git config --global user.name ...` e `user.email ...`: gravam o seu nome e e-mail para aparecerem em cada commit. O `--global` vale para todos os projetos deste computador,

então só é preciso fazer uma vez.

- `README.md` : a "capa" do repositório. O GitHub mostra esse arquivo na página inicial. A extensão `.md` quer dizer *Markdown*: `#` vira título, `-` vira lista, e os asteriscos duplos `**` deixam o texto em **negrito**.
- `.gitignore` : lista de arquivos que o Git deve **ignorar**. Aqui ficam arquivos que o sistema operacional cria sozinho (`.DS_Store` , `Thumbs.db`), arquivos de configuração do editor e, o mais importante, arquivos de segredos (`.env`), que nunca devem ir para um repositório público.
- `git init` : transforma a pasta em um repositório (cria uma pasta escondida `.git` com o histórico).
- `git branch -M main` : chama a linha principal de histórico de `main` . Uma *branch* é uma linha de histórico; vamos usar várias no Dia 14.
- `git add .` : separa **todas** as mudanças (o ponto significa "tudo") para entrarem no próximo commit.
- `git commit -m "mensagem"` : grava o commit com a mensagem. Escreva no imperativo e em português ("Cria", "Adiciona", "Corrige").
- `git remote add origin URL` : diz ao Git qual é o endereço do repositório no GitHub. `origin` é só o apelido.
- `git push -u origin main` : envia os commits para o GitHub. O `-u` lembra o destino, então nas próximas vezes basta `git push` .
- `git clone URL` : baixa uma cópia completa do repositório (com o histórico) para o seu computador.
- `git pull` : traz os commits novos do GitHub. Faça sempre antes do `git push` quando mais gente mexe no mesmo repositório.
- **Kanban**: o quadro deixa visível quem faz o quê e evita duas pessoas fazendo a mesma tarefa.

Validação

1. No terminal, digite `git log --oneline` . Deve aparecer pelo menos um commit seu, com a mensagem em português.
2. Abra a página do repositório no GitHub: devem aparecer `README.md` e `.gitignore` , e o README deve listar o nome de cada integrante já presente.

3. Abra a aba **Projects** (em português: **Projetos**): o quadro tem pelo menos 15 cartões, um para cada requisito Essencial.
4. No VS Code, o botão **Go Live** (em português: **Ir ao vivo**) aparece no canto inferior direito.

Erros comuns

1. *Mensagem:* `fatal: not a git repository` . *Causa:* o terminal está em uma pasta que não é a do projeto. *Correção:* confira que a pasta `vitrine-col` está aberta no VS Code e abra o terminal pelo menu **Terminal > New Terminal** (em português: **Terminal > Novo Terminal**) depois de abrir a pasta.
2. *Mensagem:* `error: failed to push some refs ... Updates were rejected` . *Causa:* alguém enviou commits antes de você. *Correção:* rode `git pull` e depois `git push` de novo.
3. *Mensagem:* `Author identity unknown` ou `Please tell me who you are` . *Causa:* o Passo 2 não foi feito. *Correção:* rode os dois comandos `git config --global ...` do Passo 2 e repita o `git commit` .
4. *Sintoma:* o commit foi feito, mas o arquivo não aparece no GitHub. *Causa:* faltou o `git push` . *Correção:* rode `git push` .

Se travar

1. Releia a mensagem de erro do terminal com calma: quase sempre ela diz o que falta.
2. Compare o que você digitou com o comando da aula, letra por letra (principalmente o endereço do repositório).
3. Use `git status` para ver o que o Git está vendo. Se você mexeu em um arquivo por engano, desfça com `git restore nome-do-arquivo` (isso volta o arquivo ao último commit).
4. Só depois peça ajuda à sua equipe, colando a mensagem de erro exata e dizendo o que você já tentou.

Seu projeto agora tem

- `README.md` e `.gitignore` na pasta `vitrine-col` , versionados e enviados ao GitHub.
- Um commit de cada integrante no histórico.
- Um quadro Kanban com o backlog.
- O VS Code com o Git e a extensão Live Server funcionando.

Como saber que deu certo: o repositório no GitHub mostra o README com o nome de todas as integrantes e `git log --oneline` mostra os commits de cada uma.

Capítulo 03: Aula 3 – HTML: estrutura da página, textos, links e imagens, com o Live Server

Origem: dia01-aula03-html-estrutura-da-pagina-textos-links-e-imagens.md

Aula 3 – HTML: estrutura da página, textos, links e imagens, com o Live Server

Dia 1 · Ter 06/10/2026 · Aula 3 · UC3

- **Requisitos cobertos:** não se aplica (base de todas as telas)
- **Tempo:** 10 minutos de abertura, 40 de prática e 10 de fechamento (60 minutos)
- **Ponto de partida:** pasta vitrine-col aberta no VS Code, com README.md e .gitignore versionados, repositório no GitHub e a extensão Live Server instalada (Aula 2)

Se você não terminou a aula anterior, refaça os passos dela (estão na seção Mão na Massa) antes de começar esta.

Objetivo da aula

Em 60 minutos você vai escrever a sua primeira página em **HTML**, com título, parágrafos, um link e uma imagem com texto alternativo, abri-la com o **Live Server** e versioná-la com um commit.

Abertura (10 minutos). Retomada da Aula 2: a pasta `vitrine-col` já é um repositório. Abra o VS Code, abra a pasta e rode `git status` no terminal: a mensagem deve dizer `nothing to commit, working tree clean` (nada para enviar). Hoje o repositório ganha as primeiras páginas do sistema.

O Conceito

Termos desta aula

- **HTML:** a linguagem que descreve **o que** existe em uma página (títulos, textos, imagens, links). Não é programação: é uma marcação.
- **Tag:** uma "etiqueta" entre `<` e `>` que diz o que o conteúdo é. Quase sempre vem em par: `<p>` abre um parágrafo e `</p>` fecha.
- **Atributo:** uma informação extra dentro da tag, no formato `nome="valor"`, como o `src="foto.jpg"` de uma imagem.
- **Live Server:** extensão do VS Code que abre a página em um endereço local (`http://127.0.0.1:5500`) e **atualiza o navegador sozinho** toda vez que você salva o arquivo.

Analogia: uma página HTML é como um documento do Word em que, em vez de clicar em "título" ou "negrito", você escreve etiquetas ao redor do texto. O navegador lê as etiquetas e desenha a página.

O esqueleto de toda página tem sempre as mesmas partes: `<!DOCTYPE html>` (avisa que é HTML moderno), `<html lang="pt-BR">` (a página é em português do Brasil, o que ajuda leitores de tela), `<head>` (informações que não aparecem na página, como a codificação e o título da aba) e `<body>` (tudo o que aparece).

Mão na Massa

Passo 1: crie a imagem de exemplo

A imagem abaixo é um arquivo de texto (um SVG, que é um desenho descrito em código). Crie a pasta `imagens` na raiz do projeto: no painel **Explorer** (em português: **Explorador**), clique em **New Folder** (em português: **Nova Pasta**) e digite `imagens`. Dentro dela crie o arquivo `sem-foto.svg` com este conteúdo:

Arquivo: `imagens/sem-foto.svg` (arquivo novo, inteiro)

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 600 750" role="img" aria-label="Produto sem foto">
  <rect width="600" height="750" fill="#ece7ea"/>
  <rect x="150" y="250" width="300" height="220" rx="16" fill="none" stroke="#5d5365" stroke-width="8"/>
  <circle cx="230" cy="320" r="28" fill="#5d5365"/>
  <path d="M170 450 L260 360 L330 420 L380 370 L430 450 Z" fill="#5d5365"/>
  <text x="300" y="560" font-family="sans-serif" font-size="32" text-anchor="middle" fill="#2b2230">Sem foto</text>
</svg>
```

Passo 2: crie a primeira página

Na raiz do projeto crie o arquivo `index.html` e cole o conteúdo abaixo. Salve com Ctrl+S (no Mac, Cmd+S).

Arquivo: `index.html` (arquivo novo, inteiro)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>VitrineCol</title>
</head>
<body>
  <h1>VitrineCol</h1>
  <p>Roupas das lojas do seu bairro, num só lugar.</p>
  <p>Aqui você vai poder escolher peças de várias lojas e combinar cada pedido pelo WhatsApp.</p>

  

  <p><a href="https://github.com" target="_blank" rel="noopener">Conheça o GitHub, onde guardamos o código</a></p>
</body>
</html>
```

Passo 3: abra com o Live Server

1. Com o `index.html` aberto no editor, clique no botão **Go Live** (em português: **Ir ao vivo**) no canto inferior direito do VS Code. Se preferir, clique com o botão direito no arquivo e escolha

Open with Live Server (em português: **Abrir com Live Server**).

2. O navegador abre em um endereço parecido com `http://127.0.0.1:5500/index.html` e mostra a página.
3. **Teste a atualização automática:** volte ao VS Code, troque o texto de um parágrafo, salve e olhe o navegador: ele atualiza sozinho.

Passo 4: faça experimentos curtos

Faça uma mudança de cada vez e observe o resultado no navegador:

1. Troque `<h1>` por `<h2>` (e o `</h1>` por `</h2>`): o título fica menor. Volte para `h1`.
2. Apague o valor do `alt` da imagem e salve. Nada muda na tela, mas um leitor de tela deixaria de descrever a imagem. Coloque o texto de volta.
3. Troque o endereço do `src` por `imagens/nao-existe.svg`: a imagem quebra e aparece o texto do `alt`. Volte para `imagens/sem-foto.svg`.

Passo 5: faça o commit da aula

```
git add .
git commit -m "Cria a primeira página e a imagem de exemplo"
git push
```

Explicação do Código

index.html

- `<!DOCTYPE html>`: primeira linha de toda página. Diz ao navegador que o arquivo usa o HTML atual.
- `<html lang="pt-BR">`: abre o documento e declara o idioma. Fecha na última linha (`</html>`).
- `<head> ... </head>`: parte "de bastidores".
- `<meta charset="UTF-8">`: define a codificação dos caracteres. Sem ela, acentos como "ã" e "ç" podem aparecer errados.

- `<meta name="viewport" content="width=device-width, initial-scale=1">` : faz a página se ajustar à largura do celular. Vamos usar isso no Dia 5.
- `<title>VitrineCol</title>` : texto que aparece na aba do navegador.
- `<body>` ... `</body>` : tudo o que a pessoa vê.
- `<h1>` : o título principal. Cada página deve ter **um único** `h1` .
- `<p>` : um parágrafo.
- `` : uma imagem. `src` é o caminho do arquivo, `alt` é o texto que descreve a imagem (para quem não enxerga e para quando a imagem não carrega), `width` e `height` reservam o espaço da imagem para a página não "pular" ao carregar. A tag `img` não tem fechamento.
- `<a href="..." target="_blank" rel="noopener">` : um link. `href` é o destino; `target="_blank"` abre em uma nova aba; `rel="noopener"` é uma proteção de segurança para links que abrem em nova aba.

sem-foto.svg: um desenho feito com retângulo, círculo e um caminho (`path`), mais o texto "Sem foto". Será a imagem mostrada quando um produto não tiver foto.

Validação

1. A página abre pelo Live Server, com título, dois parágrafos, a imagem e o link.
2. A aba do navegador mostra o texto "VitrineCol".
3. Ao salvar uma mudança no arquivo, o navegador atualiza sozinho.
4. `git log --online` mostra o novo commit, e o GitHub mostra `index.html` e a pasta `imagens` .

Erros comuns

1. *Sintoma*: acentos aparecem estranhos (por exemplo `Ã§`). *Causa*: faltou a linha `<meta charset="UTF-8">` ou o arquivo foi salvo em outra codificação. *Correção*: confira a linha e salve como UTF-8 (no canto inferior direito do VS Code aparece **UTF-8**).
2. *Sintoma*: a imagem aparece quebrada, com um ícone de página rasgada. *Causa*: o caminho do `src` está errado ou a pasta/arquivo tem outro nome. *Correção*: confira se a pasta se chama `imagens` e o arquivo `sem-foto.svg` , tudo em minúsculas, sem espaços.

3. *Sintoma*: parte da página sumiu ou ficou toda em negrito. *Causa*: uma tag foi aberta e não foi fechada (por exemplo `<p>` sem `</p>`). *Correção*: confira se cada tag que abre tem a sua que fecha.
4. *Sintoma*: o botão **Go Live** (em português: **Ir ao vivo**) não aparece. *Causa*: a extensão Live Server não está instalada ou nenhuma pasta está aberta. *Correção*: refaça o Passo 1 da Aula 2 e abra a pasta `vitrine-col`.

Se travar

1. Aperte F12 no navegador e olhe a aba **Console** (em português: **Console**): erros de arquivo não encontrado aparecem lá em vermelho.
2. Compare o seu `index.html` com o da aula, linha por linha.
3. Veja o que mudou com `git status`; se estragou o arquivo, volte ao último commit com `git restore index.html`.
4. Só depois peça ajuda à sua equipe, dizendo o arquivo, a linha e o que você já tentou.

Seu projeto agora tem

- `README.md`, `.gitignore` (Aula 2).
- `index.html`: uma página simples (será reescrita nas próximas aulas).
- `imagens/sem-foto.svg`: a imagem padrão de produto sem foto.
- O Live Server abrindo a página e atualizando ao salvar.

Como saber que deu certo: você troca um texto no `index.html`, salva, e o navegador mostra a mudança sem você apertar nada.